Reflexión sobre el trabajo de integración

Metodología I · Tecnicatura Universitaria en Programación, UTN FRM · Proyecto: sistema de gestión de turnos odontológicos

15 de octubre de 2026
Pagano Amanda

Introducción

Desarrollé un sistema de gestión de turnos odontológicos trabajando con SDD, Active Stack y Claude Code, y avancé hasta el change C-02, que elegí para analizar en este trabajo.
Empecé con un repositorio vacío, recién creado con git init, e inicialicé /active-orchestrator, que me fue guiando por el Discovery, la KB, el roadmap, las skills y las reglas del agente.
Después implementé C-01, que preparó el proyecto Python, las herramientas de test y la regla de que el dominio no dependiera de nada externo. Esa base me permitió avanzar con C-02, el primer change con reglas de negocio, que valida que no haya turnos solapados por profesional o por sillón.

1. Información del Discovery que resultó incorrecta o no verificable, y cómo la detecté

Una vez inicializado /active-orchestrator, primero se creó el Discovery (la investigación de mercado), y después tuve que corregirlo porque tenía datos erróneos. Ahí vi la importancia del humano en combinación con la IA. Si bien el agente no inventó productos ni fuentes, ya que todos los sistemas y todas las URLs existen, sí afirmó cosas que las fuentes no decían: exageró, completó huecos o atribuyó datos a la fuente equivocada. Por ejemplo:

- Odonthia: el informe decía que este "impide" los solapamientos; su centro de ayuda dice que avisa y deja forzar el sobreturno. Cambiaba la conclusión sobre el diferencial del producto.
- DentalCore: este había quedado segundo en el ranking cuando es un producto que todavía no se lanzó al mercado.
- Bilog: la exportación de los datos era opcional, no obligatoria. AgendaPro: tenía precios atribuidos a una fuente equivocada.
- Dentalink: el informe le daba 3,05 de puntaje, pero el script calculaba 3,00.

En los casos de Odonthia, Bilog y AgendaPro los detecté verificando a mano las fuentes: abrí cada página y comparé lo que decía con lo que afirmaba el informe. El de DentalCore lo noté revisando el ranking. Y el error de Dentalink lo encontré comparando el puntaje del script con el del informe.

2. Dónde usé Ajustar, qué estaba mal y qué habría pasado con Continuar

En la base de conocimiento (KB), los 10 archivos que reúnen las reglas de negocio, el modelo de datos y las decisiones tomadas a partir del Discovery, también hubo ajustes por hacer: figuraba el plazo anterior y el primer cambio propuesto era demasiado grande. Si apretaba «Continuar» sin leer, revisar y chequear todo a mano, el proyecto iba a quedar lleno de inconsistencias.

- Discovery: la sección D.3 no reflejaba mis decisiones y faltaban el resumen ejecutivo, las fechas y las referencias; por ende, la KB y el roadmap se habrían armado sobre un MVP desactualizado.
- Skills: la opción "para otros agentes" creó 52 carpetas de más; antes de borrarlas hice una simulación con git clean -n.

Si seguía con «Continuar» y hacía el commit, se habrían subido todos los errores al repositorio: decenas de carpetas de herramientas que no eran necesarias, generando un repositorio lleno de ruido, donde sería muy difícil encontrar los entregables y navegar dentro de él. Y aunque las borrara después, habrían quedado guardadas en el historial de Git.

3. Diferencia entre implementar con una especificación aprobada y pedir código con un prompt suelto

Aquí entendí la importancia de un prompt bien redactado, y no solo de ideas sueltas. Pude notar cómo en horas se crea un proyecto que normalmente tardaría días. Con un prompt suelto quedan cosas sin aclarar y la IA puede resolverlas según su propia interpretación.

En C-02, por ejemplo, tomé 7 decisiones antes de empezar a programar. La especificación tuvo 31 escenarios, incluidos el caso feliz, el error de negocio y el caso borde. Cada escenario se cubrió con al menos un test: 107 en total. Eso me permitió comprobar el comportamiento del sistema con criterios que había definido previamente.

También hubo momentos en los que tuve que revisar antes de seguir. En la tarea 4.3, el agente frenó para que revisara los tipos y la API antes de implementar las reglas. En C-01, cuando el código se apartó del diseño en D7, esa diferencia quedó registrada en el design.md.

Otra ventaja que vi fue cuando en clase le preguntamos al profe qué lenguaje teníamos que usar. Yo había elegido TypeScript, pero finalmente la cátedra definió que se trabajara con Python. Como todavía no había empezado a escribir código, solo tuve que actualizar los documentos. Ahí entendí que revisar y tomar decisiones antes de implementar puede ahorrar bastante trabajo después.

4. Qué parte de la fundación le aportó más valor al ciclo OPSX, y cuál menos

Creo que la parte "vital" de este proyecto fue el AGENTS.md, con reglas claras que el agente leyó y siguió al pie de la letra, por más que las skills, por ejemplo, le "dijeran" lo contrario.
Un ejemplo concreto fue lo que pasó con las skills de testing: proponían usar mocks y raise ValueError, pero para el dominio se respetaron las reglas que había definido en el AGENTS.md. Eso me dio una base para revisar que el código siguiera los criterios del proyecto.
Engram es una genialidad: soluciona el bache de la memoria volátil de las IA con una memoria persistente que guarda el contexto entre sesiones. Aunque no guarda todo: registra las decisiones pero no siempre el motivo. Además, en las memorias #9 y #13 vi que las fechas podrían resultar confusas al actualizarse, y esa memoria queda en mi máquina, no en el repositorio.
La parte que menos valor me aportó en este ciclo fueron las fases del roadmap que todavía no llegué a usar. Me sirvieron para tener una idea de cómo puede continuar el proyecto, pero no tuvieron tanta influencia en lo que implementé hasta ahora. Si el proyecto sigue, seguramente van a tener más utilidad para organizar los próximos pasos.

5. Uso de IA

Me ayudé bastante con Claude (el chat) a la vez que trabajaba con Claude Code, para entender conceptos, ajustar prompts o tomar decisiones en las que, en ocasiones, no estaba tan segura.
Claude Code con Active Stack generó el Discovery, la KB, el roadmap, las especificaciones y el código. Mientras tanto, el chat de Claude me fue guiando paso a paso y me ayudó a entender lo que estaba haciendo. Yo revisé lo que se generaba y decidí qué aceptar, qué corregir y cuándo continuar. También verifiqué manualmente las fuentes, corrí los verify, no acepté permisos amplios, hice una simulación antes de borrar, hice los commits y corrí los tests a mano antes de archivar, para comprobar que todo funcionara como esperaba.